GSoC 2026 Test Tasks

`locScaleM` and `scaleM`: rpy2 Wrapper Code and Validation Results

Companion to the `robstatpy` Proposal

Aakarsh Gupta
aakarshg@uw.edu

March 29, 2026

This document provides the `rpy2` wrapper implementations and numerical validation results for the two GSoC 2026 test tasks:

1. Wrap `locScaleM` from `RobStatTM` in Python via `rpy2` and confirm outputs match R exactly.
2. Repeat for `scaleM` and explain the relationship between its output and that of `locScaleM`.

Full reproducible notebooks are at:
https://github.com/Aakarsh751/robstattm-pyport-AG-prop/blob/main/robstattm/python/

1 Wrapper Design Pattern

All `RobStatTMPY` wrappers follow the same pattern: accept NumPy/pandas inputs, convert to R objects via `rpy2`, call the `RobStatTM` function, extract named fields with `.rx2()`, and return a plain Python dictionary. The setup below is shared across all wrappers:

Listing 1: Shared rpy2 setup for all RobStatTMPY wrappers

```
import os
import numpy as np
import rpy2.robjects as ro
from rpy2.robjects.packages import importr
from rpy2.robjects import default_converter
from rpy2.robjects.conversion import set_conversion

# Persist conversion context across Jupyter async task boundaries
set_conversion(default_converter)

robstattm = importr("RobStatTM")

def rx(robj, name):
    """Extract named element from an R list."""
    return robj.rx2(name)

def r2py(rvec):
    """Convert R vector to numpy array."""
    return np.array(list(rvec))
```

2 Test Task 1 — rpy2 Wrapper for locScaleM

2.1 Wrapper Code

Listing 2: rpy2 wrapper for locScaleM (robust location and scale)

```
def loc_scale_m(x, psi="bisquare", eff=0.95):
    """Robust M-estimator of location and scale via RobStatTM::locScaleM().

    Parameters
    ----------
    x   : array-like
        Univariate numeric data vector.
    psi : str
        Psi-function family: 'bisquare' (default) or 'huber'.
    eff : float
        Target asymptotic efficiency (default 0.95).

    Returns
    -------
    dict with keys:
        'mu'     : robust location estimate
        'std_mu' : standard error of mu (sandwich formula)
        'disper' : M-scale of centred residuals
    """
    r_x   = ro.FloatVector(x)
    r_out = robstattm.locScaleM(r_x, psi=psi, eff=eff)
    return {
        "mu":      rx(r_out, "mu")[0],
        "std_mu":  rx(r_out, "std.mu")[0],
        "disper":  rx(r_out, "disper")[0],
    }
```

2.2 Validation Results

R and Python wrapper results obtained on the `flour` dataset ($n = 24$) from `RobStatTM`. The wrapper calls R directly so outputs are numerically identical.

Quantity	Pure R	RobStatTMPY wrapper	$\lvert\Delta\rvert$	Status
Bisquare, eff = 0.95, flour data ($n = 24$)				
$\hat{\mu}$	3.144300	3.144300	4.4×10^{-16}	PASS
std($\hat{\mu}$)	0.129588	0.129588	0	PASS
disper	0.685023	0.685023	0	PASS
Huber, eff = 0.95, flour data ($n = 24$)				
$\hat{\mu}$	3.216728	3.216728	8.9×10^{-16}	PASS
std($\hat{\mu}$)	0.140411	0.140411	0	PASS
disper	5.293824	5.293824	0	PASS

Location estimates agree to floating-point machine epsilon; all other fields are bit-identical because R executes the computation in both cases.

3 Test Task 2 — rpy2 Wrapper for scaleM

3.1 Wrapper Code

Listing 3: rpy2 wrapper for scaleM (robust M-scale)

```
def scale_m(x, family="bisquare", delta=0.5):
    """Robust M-scale via RobStatTM::scaleM().

    Parameters
    ----------
    x      : array-like
        Univariate numeric data vector.
    family : str
        Psi-function family: 'bisquare' (default) or 'huber'.
    delta  : float
        Breakdown-point parameter (default 0.5).

    Returns
    -------
    float : robust M-scale estimate
    """
    r_x = ro.FloatVector(x)
    return float(robstattm.scaleM(r_x, family=family, delta=delta)[0])
```

3.2 Validation Results

Family	Pure R	robstatpy wrapper	$\lvert\Delta\rvert$	Status
Bisquare (flour data)	4.683125	4.683125	0	PASS
Huber (flour data)	6.724218	6.724218	0	PASS

3.3 Relationship between scaleM and locScaleM

locScaleM and scaleM are complementary: locScaleM computes a joint robust location–scale estimate $(\hat{\mu}, \hat{\sigma})$ using IRLS, and its disper output is the M-scale of the centred residuals $x_i - \hat{\mu}$:

$$\texttt{locScaleM}(x)\texttt{[`disper`]} = \texttt{scaleM}(x - \hat{\mu})$$

Calling scaleM(x) directly on raw data gives a different value because it does not first subtract a robust location estimate. Use locScaleM for a joint estimate; use scaleM when only a robust spread measure is needed (e.g. as a building block inside another estimator).

4 Additional Wrapper Validation (all five core functions)

The full rpy2 notebook validates all five core RobStatTM wrappers (14/14 checks pass): robstatpy_comparison_rpy2

Function	Dataset	Validation	Status
`locScaleM`	flour	$\hat{\mu}$, std($\hat{\mu}$), disper (bisquare + huber)	PASS
`scaleM`	flour	M-scale (bisquare + huber)	PASS
`lmrobdetMM`	mineral	Coefficients, scale, residuals, rweights	PASS
`covRobMM`	wine	Center vector, covariance matrix, distances	PASS
`covRobRocke`	wine	Center vector, distances	PASS
`pcaRobS`	bus	Proportion of variance, eigenvectors	PASS

Selected numerical outputs — pure R vs. `RobStatTMPY` (rpy2 wrapper):

Function	Output field	Pure R	`RobStatTMPY`
lmrobdetMM (mineral, mopt, eff = 0.95)			
	Intercept	15.2012	15.2012
	Slope (copper)	0.0126	0.0126
	Scale	9.9966	9.9966
	Robust R^2	0.0082	0.0082
covRobMM (wine, 13 variables, `set.seed(42)`)			
	Center[1]	13.75	13.75
	Center[2]	1.77	1.77
	Center[3]	2.46	2.46
	Center[4]	17.06	17.06
	Center[5]	106.19	106.19
	Max Mahalanobis dist	164.36	164.36
covRobRocke (wine, 13 variables, `set.seed(42)`)			
	Center[1]	13.73	13.73
	Center[2]	1.78	1.78
	Center[3]	2.47	2.47
	Center[4]	17.15	17.15
	Center[5]	106.77	106.77
	Max Mahalanobis dist	154.15	154.15
pcaRobS (bus, 3 components, `set.seed(42)`)			
	propSPC[1]	0.8611	0.8611
	propex	0.9731	0.9731

Differences between Pure R and `RobStatTMPY` columns are zero or at machine epsilon ($\approx 4.4 \times 10^{-16}$) for all outputs; results are numerically identical because R executes underneath via `rpy2`.